

Section 8

Week 10 — Backup & Recovery

From Last Week

We covered two things:

- **WAL** → automatic crash protection. PostgreSQL handles it. You don't touch it.
- **pg_dump** → manual snapshot. You run it. It creates a `.sql` file you can restore later.

We ended with this question: *"You take a pg_dump at 2:00 AM. The disk fails at 4:00 AM. What do you lose?"*

Two hours of data. Everything that happened between the backup and the failure is gone — because it wasn't in the backup file.

That's the problem we solve today.

1. The Backup Gap

```
2:00 AM → pg_dump taken (snapshot frozen here)
```

```
2:01 AM → 1,000 orders inserted
```

```
3:30 AM → 500 payments processed
```

```
4:00 AM → DISK FAILURE
```

```
Restore from 2:00 AM backup:
```

```
→ 1,000 orders: GONE
```

```
→ 500 payments: GONE
```

```
→ Total data lost: 2 hours of real business activity
```

`pg_dump` is a **point-in-time snapshot**. It literally does not know anything happened after it ran.

The question is: can we do better?

Yes — by combining a full backup with a continuous stream of every change after it. That's exactly what **WAL archiving + PITR** provides.

2. Full vs Incremental Backup

Full Backup

Copies everything — every table, every row, every index, every sequence.

Property	Details
Self-contained	Yes — can restore without anything else
Size	Large — the entire database
Restore time	Fast — apply one thing
Tools	<code>pg_dump</code> (logical), <code>pg_basebackup</code> (physical)

Incremental Backup

Copies only what **changed** since the last backup.

Monday 2AM

→ Full backup

(10 GB – the base)

Tuesday 2AM

→ Incremental

(180 MB – only Monday→Tuesday changes)

Wednesday 2AM

→ Incremental

(210 MB – only Tuesday→Wednesday changes)

Thursday 10AM

→ SERVER DIES

Recovery:

Step 1: Apply Monday full backup

Step 2: Apply Tuesday incremental

Step 3: Apply Wednesday incremental

→ Database at Wednesday 2AM state. Lost: ~8 hours.

Property	Details
Self-contained	No — needs full + all incrementals in order
Size	Small — only changed data
Restore time	Slower — must apply each piece sequentially
Tools	WAL archiving (PostgreSQL), <code>pgBackRest</code> , <code>Barman</code>

For this course: The important thing is the concept. In PostgreSQL, WAL segments *are* the incremental layer. Each 16MB WAL file is a chunk of change history.

The Two Backup Tools

	<code>pg_dump</code>	<code>pg_basebackup</code>
What it copies	SQL statements	Raw binary files

	pg_dump	pg_basebackup
Output size	Smaller	Larger (~1–2 GB)
Restores to	That exact moment only	Any second after the backup
Use case	Clone a DB, move to staging	Full disaster recovery
You've used it?	Yes — to create hospital_fast	Just now in this session

3. pg_basebackup — Physical Backup

What it is

pg_dump outputs SQL statements. pg_basebackup copies the **raw binary data files** — the actual files PostgreSQL uses internally. It's a complete copy of the entire database cluster.

pg_dump vs pg_basebackup — side by side

	pg_dump (Logical)	pg_basebackup (Physical)
Output	SQL statements	Raw binary data files
Portable across versions	✔ Yes	✗ No — same major version
Supports PITR	✗ No	✔ Yes (with WAL archiving)
Restore speed	Slower (replays SQL)	Faster (copies files)
Single table restore	✔ Yes	✗ No — full cluster only
Human-readable	✔ Yes	✗ No — binary
Use case	Migration, partial restore, staging	Full disaster recovery with PITR

The command

● ● ●

```
# Set PATH first

$env:PATH = "C:\Program Files\PostgreSQL\18\bin;" + $env:PATH


# Take a full physical backup

pg_basebackup -U postgres -D C:\hospital_physical_backup -Fp -Xs -P
```

Flag	Meaning
-D	Output directory (must NOT already exist)
-Fp	Format: plain (copies files as-is)
-Xs	Include WAL for consistency
-P	Show progress

What you'll see in the output directory: the same structure as PostgreSQL's `data/` folder — `base/` , `global/` , `pg_wal/` , config files. It looks exactly like a real cluster. Because it *is* one.

Why you can't just copy the data folder

Copying PostgreSQL's `data/` folder while it's running gives you a **corrupt backup**:

- Some pages are in memory (not yet written to disk)
- WAL hasn't been fully applied
- You'd get a half-written, inconsistent state

`pg_basebackup` handles this correctly using PostgreSQL's internal backup protocol — it takes a consistent snapshot even while transactions are happening.

5. Point-in-Time Recovery (PITR) — The Complete Picture

What PITR means

Restore the database to the **exact state it was in at any specific second in time**.

Not "the state at the last full backup." Not "the state at 2:00 AM." The state at **14:46:30** — 30 seconds before the disaster.

The three ingredients

Ingredient	What it is
Base backup	<code>pg_basebackup</code> output — full snapshot at backup time
WAL archive	All WAL files saved from backup time to "now"
Recovery target	The time you want to restore to

The One-Sentence Summary

`pg_dump` = photograph. `pg_basebackup` + WAL = time machine.

You won't configure this yourself — that's a DBA's job.

But you need to know it exists and why it matters.

Next Week

We build the project together.

Before next week: **set up your two databases** (`hospital_slow` and `hospital_fast`).